



Data analysis with AI and vibe coding

2026. 09. 18.

National Institute of Animal Science
Smart Livestock Environment Division
Jang Dong-hwa, Agricultural Researcher



1 AI overview

2 Training an image detection model

3 Understanding vibe coding

4 Research applications



Step 1

AI

overview

Intro

We use AI daily — but how well do we understand it?

Familiar – AI as part of daily work

Analysis code, graphs, summaries — all from a single prompt

Writing code

Fixing errors

Summarizing
papers

Drafting reports

Drawing graphs

Question

We apply AI to research —
but do we understand how
it works?



Map

How are AI, machine learning and deep learning related?

AI (artificial intelligence)

Technology performing intelligent tasks such as perception and reasoning

Machine learning

Methods that learn patterns from data

Deep learning

Methods that learn with multi-layer neural networks

Feature

Machine learning

- ✓ Whether rules are hand-written or learned from data
- ✓ Learns decision rules from data

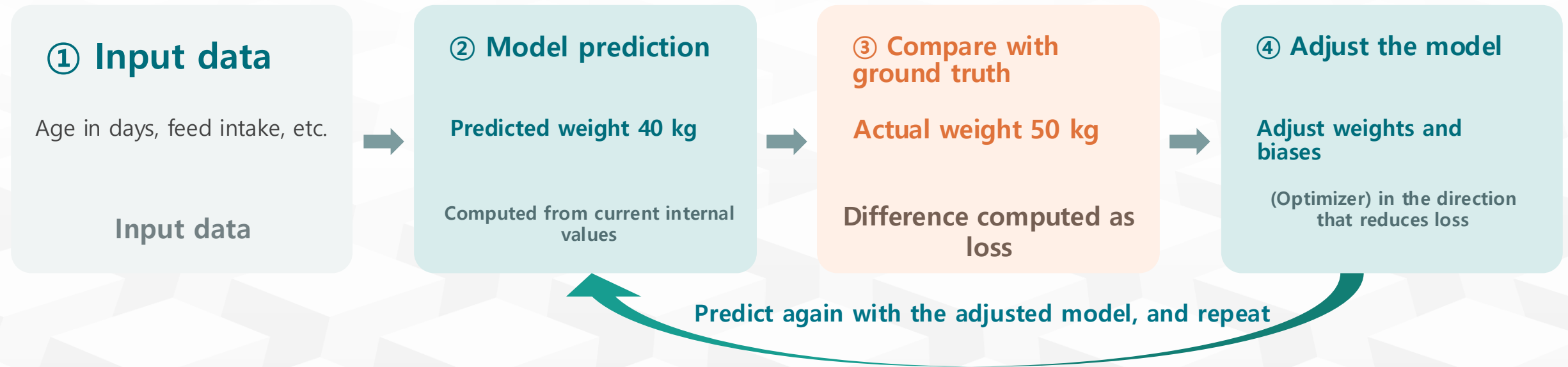
Deep learning

- ✓ Whether representations are learned stage by stage through multiple layers
- ✓ Learns features and relations with multi-layer neural networks

Points to note

- ✓ Not distinguished only by whether humans define the features
- ✓ No universal rule on how many layers make it deep learning

How does AI learn?



Parameters = internal model values tuned by training

Training compares predictions with ground truth and adjusts the model's internal values

AI는 어떻게 학습할까요?

학습 전

데이터와 예측

● 실제 데이터 — AI 예측



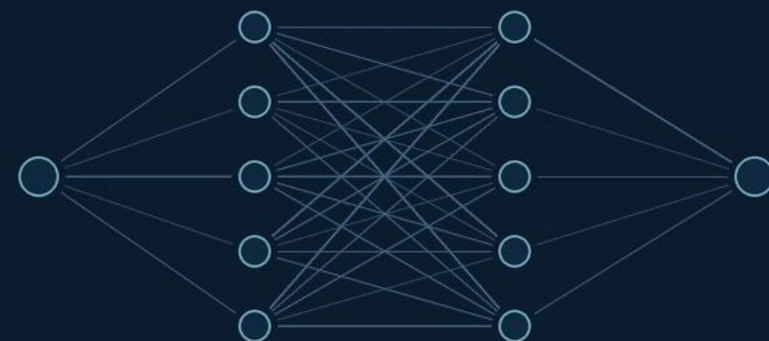
학습 횟수 0 회

학습 오차 0.2655

작을수록 좋음

신경망 내부

입력 계산 계산 예측



입력에서 예측으로 계산

선이 굵을수록 연결 강도의 크기가 큼

처음에는 예측이 데이터와 잘 맞지 않습니다.

흰 점은 실제 데이터, 만트색 선은 AI의 예측입니다.

01 예측

02 오차 확인

03 조정 방향 계산

04 내부 값 조정

Key terms

Values AI finds vs. values humans set

Define input/output

Select model

Define loss

Optimizer

Tune training settings

Internal model values

Parameters

Parameter

Weights, biases, etc.

Tuned during training
to reduce prediction error

The values changed in "Adjust the model" on the
previous slide

Training settings

Hyperparameters

Hyperparameter

Epoch

Number of passes over the full training data

Batch size

Number of samples processed together

Learning rate

Controls the update size of internal values

Set by defaults, manual choice, or automatic search

Key terms

How does training change when hyperparameters change?

General tendencies, all else equal — not rules guaranteeing better performance

Setting	Role	Increase	Decrease
Learning rate (Learning rate)	Controls the size of each update	Larger steps Too large — unstable training	Smaller steps Too small — slow training
Batch size (Batch size)	Number of samples processed at once	More memory use Less variance in gradient estimates	Less memory use More variance in gradient estimates
Epochs (Epochs)	Number of passes over the full training data	More training and time Too many — risk of overfitting	Less training and time Too few — model underlearns

Some settings are choices of type: optimizer (SGD, Adam), activation function (ReLU, etc.)

Key terms

Must we train AI from scratch every time?

Common starting point

Pretrained model

A model already trained on existing data

Take the trained weights and adapt them to your purpose

Use without further training

Keep the model weights unchanged

Feed input data or prompts
and use the model's built-in capability

e.g. enter a target as text to extract image regions

Use with fine-tuning

The main form of transfer learning

Train further on task-specific data

Update the trainable parameters
to adapt the model to new data and tasks

e.g. train a pretrained detection model on pig images

Transfer learning: reuse knowledge already learned for another task or data setting

Either way, performance must be checked on your own research data

Key terms

Checking model performance by splitting the data

Training (Train)

Practice problems

Solve problems repeatedly to learn patterns in the data

Adjusts parameters such as weights and biases

Validation

Mock exam

Use results to choose the training method and stopping point

Compare hyperparameters, select the model

Test

Final entrance exam

After fixing the model and settings, check final performance

Test results are not used to change settings

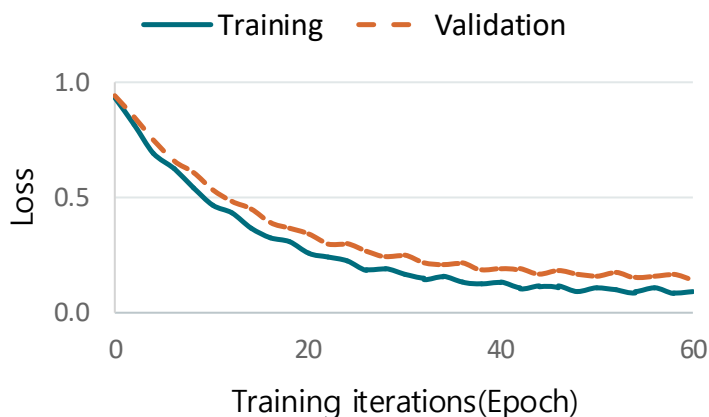
Example split: training 70% / validation 15% / test 15%

※ Ratios adjusted to data size and research purpose

Reading training status from training and validation loss curves

Loss: prediction error as a number · under the same criteria, lower is better

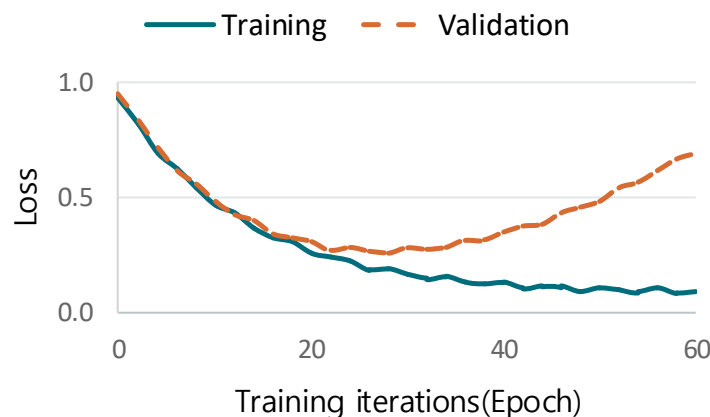
Both improving



Training and validation loss both fall

Watch whether validation performance improves

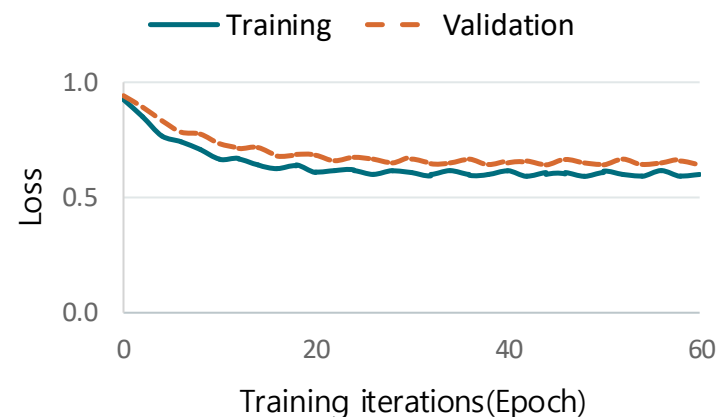
Possible overfitting



Training loss falls · validation loss rises

Based on validation performance, consider early stopping or stronger regularization

Possible underfitting or stalled training



Neither loss falls far enough

Check causes: model, learning rate, data; then adjust the training approach

※ Illustrative curves only · absolute loss or curve gap alone cannot confirm the state

Judge training state with validation data · keep test data for final evaluation

Adjusting model and training settings to reduce overfitting

Method	Adjustment	Cautions
Early stopping (Early stopping)	Stop training when validation performance stops improving	- Allow for short-term fluctuation - Keep the best-validation model
Dropout (Dropout)	Randomly zero some outputs during training; consider applying or raising the rate	- Too high a rate hinders learning - Not applied at inference
L2 regularization (L2 regularization)	Stronger penalty on large weights; consider raising the strength	- Too much regularization can cause underfitting - Check training performance as well
Simpler model (adjust depth and width)	Reduce complexity, e.g. fewer or narrower layers	- Over-reduction limits representational capacity - Check that needed patterns are still learned

Also check data volume, diversity, label quality, and training-validation differences

Verify effects on validation data · keep test data separate until final evaluation



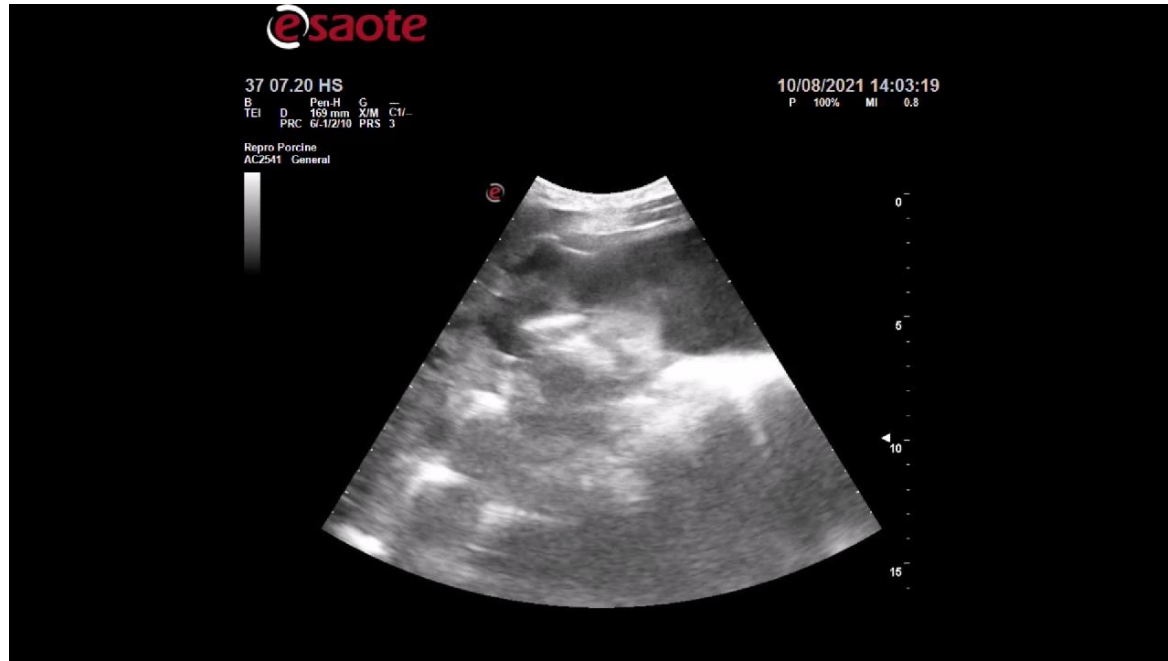
Step 2 Training an image detection model

National Institute of Animal Science is leading a new future through research and technology

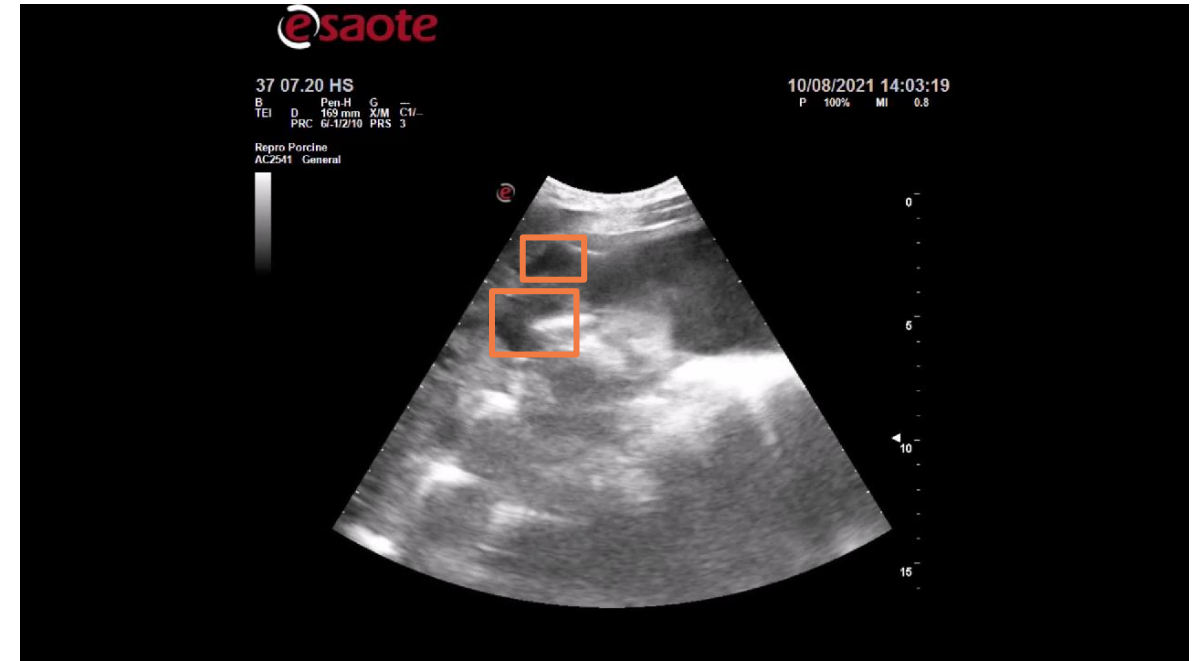
Ultrasound detection practice for pig pregnancy diagnosis

Research goal: pregnancy diagnosis · this practice: locating the labelled region of interest

Input: ultrasound image



Label: human-marked region



Same image compared · boxes at right are ground-truth labels, not model predictions

Region detection is distinct from pregnant/non-pregnant classification performance

Detection model training workflow

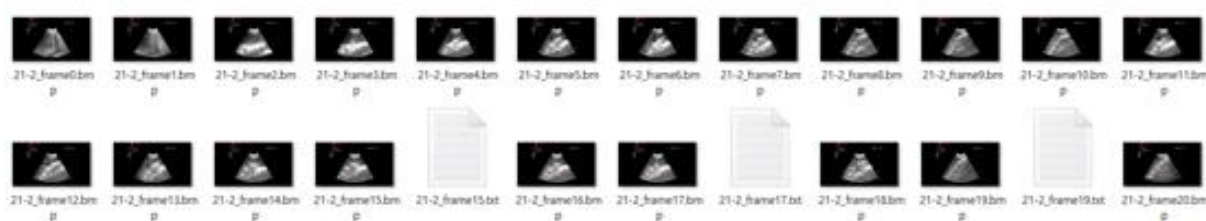
Focus on what to prepare and check at each step, not on code syntax

Step	Work	Outputs to check
1. Data preparation	Select data	Source images and capture info
2. Labelling	Define targets and bounding boxes	One label file per image
3. Data splitting	Split by animal and capture session	Training / validation / test lists
4. Training run	Set pretrained model and training options	Training logs and model weights
5. Result review	Predict new images, compare with labels	Missed and false detections, plus metrics

Actual data structure (ultrasound images and label files)

Image-to-label pairing as seen in the actual folder

Part of the data folder



Selected image



21-2_frame15
.bmp

Ultrasound image

Label file with the same name

21-2_frame15.txt

```
0 0.455078 0.384722 0.053906 0.063889
0 0.438672 0.482639 0.071094 0.098611
```

Label file recording target class and position

One line of numbers = one bounding box

Image and label are linked by the same filename, extension aside

Saving image boxes as a label file

Marking a region in the labelling tool saves class and coordinates to a file

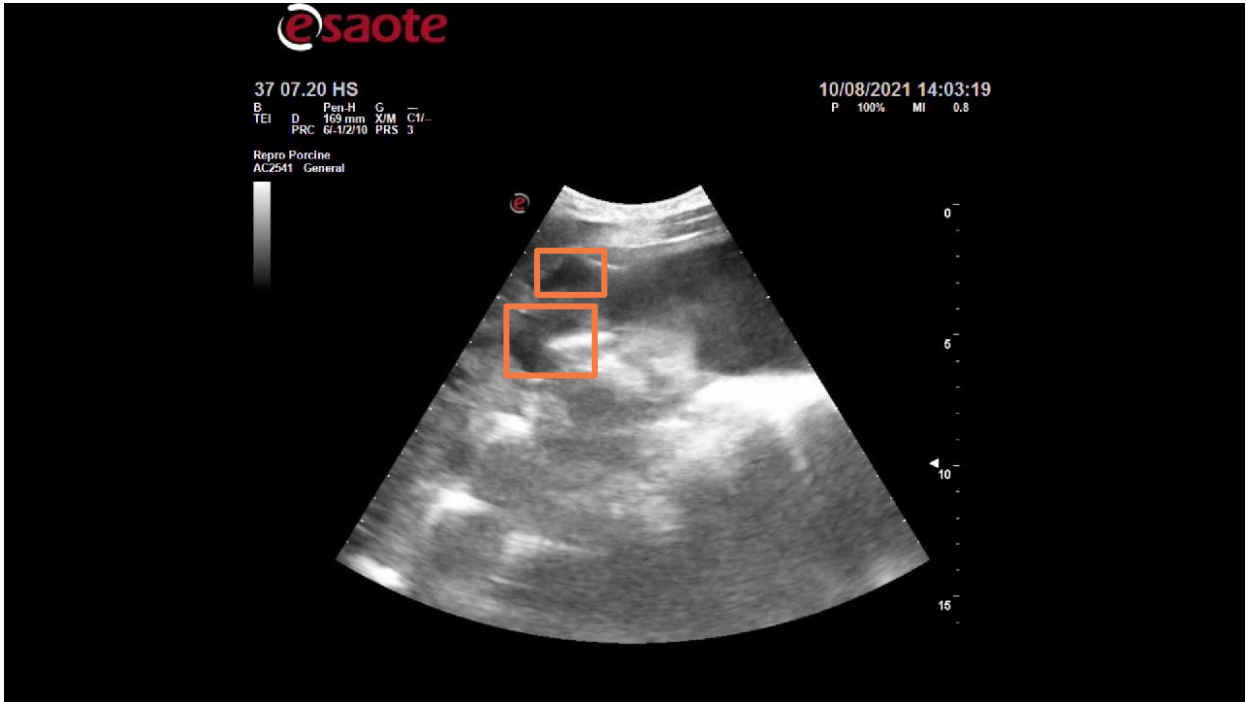


Image: 21-2_frame15.bmp

Label: 21-2_frame15.txt

```
0 0.455078 0.384722 0.053906 0.063889
0 0.438672 0.482639 0.071094 0.098611
```

Item	Meaning
First number	Class ID
Next four numbers	Centre x·y, box width·height
Coordinate basis	0–1 ratio of image size
One line in the file	One bounding box

Agree first on how to label blurred or partly visible targets

Check label status before image count

Provided data: 189 BMP images · 1 class (pregnancy) · 304 bounding boxes

Data status	Images	Check before training
Labels with boxes	119	Check for missed targets and box placement
Empty labels	13	Check if truly no target or simply not done
No matching label file	57	Check status, then label or exclude

119 + 13 = 132 images with label files
Distinguish target-free images from unlabelled images

Reflect no target and not yet labelled separately in the dataset

Keep material from the same recording out of both sets

Random splitting of consecutive frames risks evaluating on near-identical scenes

Split in the original practice

Random 8:2 split by image

Adjacent frames from the same video may appear in both training and validation

Practice results are hard to read directly as independent final performance

Split suited to the research goal

Split by video, animal or exam first

Group data from the same animal or recording, then assign groups to different sets

Folder names alone cannot prove independence
Check animal ID and capture info

Apply augmentation after splitting, to training data only · keep validation and test originals separate

Preparing image, label and config files for YOLO

Example rebuilt for Colab · uses data after splitting and label review

Location	Example file
images/train/	sample001.bmp
labels/train/	sample001.txt
images/val/ · labels/val/	Validation images and labels with the same names
images/test/ · labels/test/	Final-evaluation images and matching labels

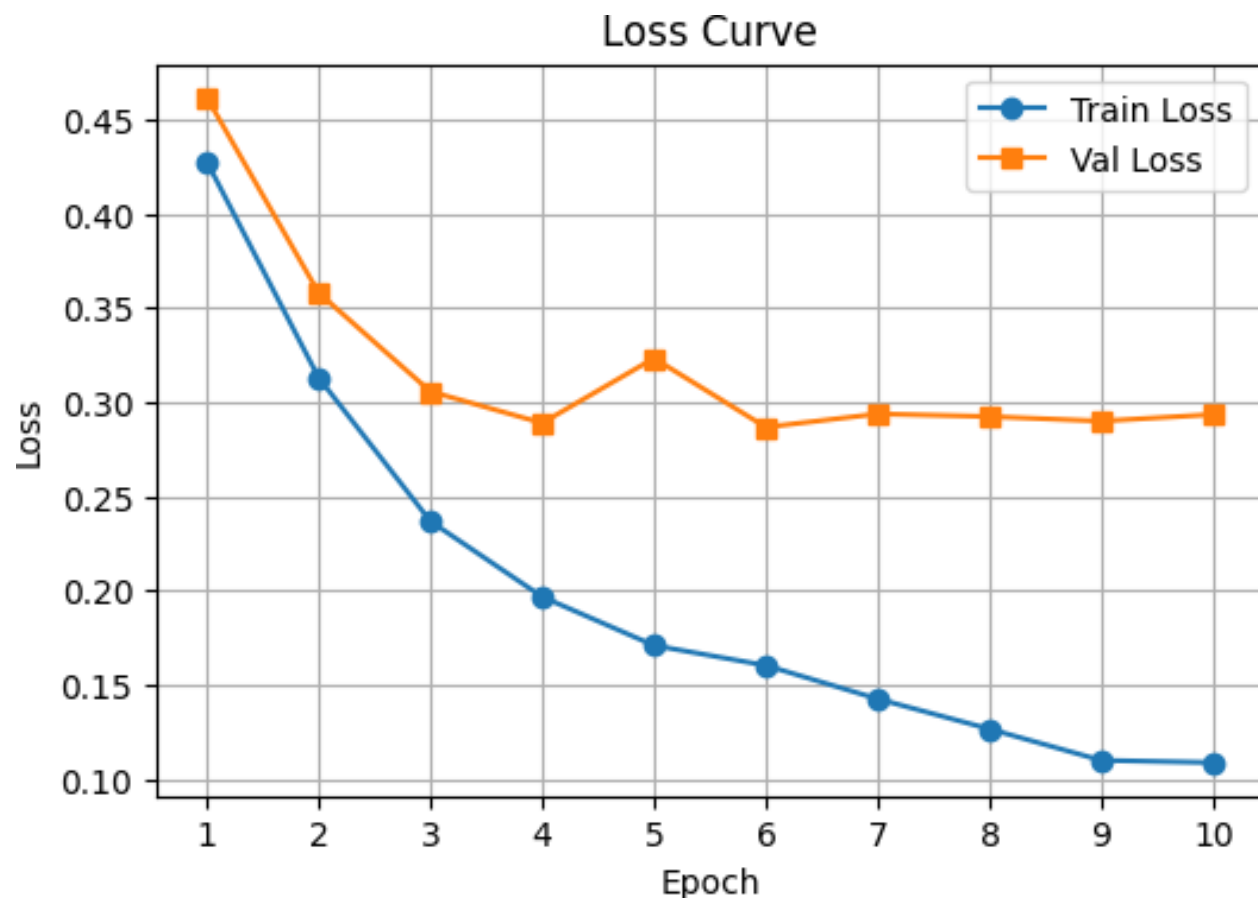
Yaml: human-readable text format for configuration

data.yaml

```
path: /content/pig_dataset
train: images/train
val: images/val
test: images/test
names:
0: pregnancy
```

Link image and label by the same name · data.yaml sets paths and classes

Validation plateau seen in a real learning curve



What the graph shows

- ✓ Train Loss: keeps decreasing
- ✓ Val Loss: drops early, then plateaus

Check for possible overfitting

- ✓ Check if more training improves validation
- ✓ Compare validation metrics with prediction results
- ✓ Save model by chosen criterion

Checklist when training does not improve

Even with no error message, check inputs, labels and training settings

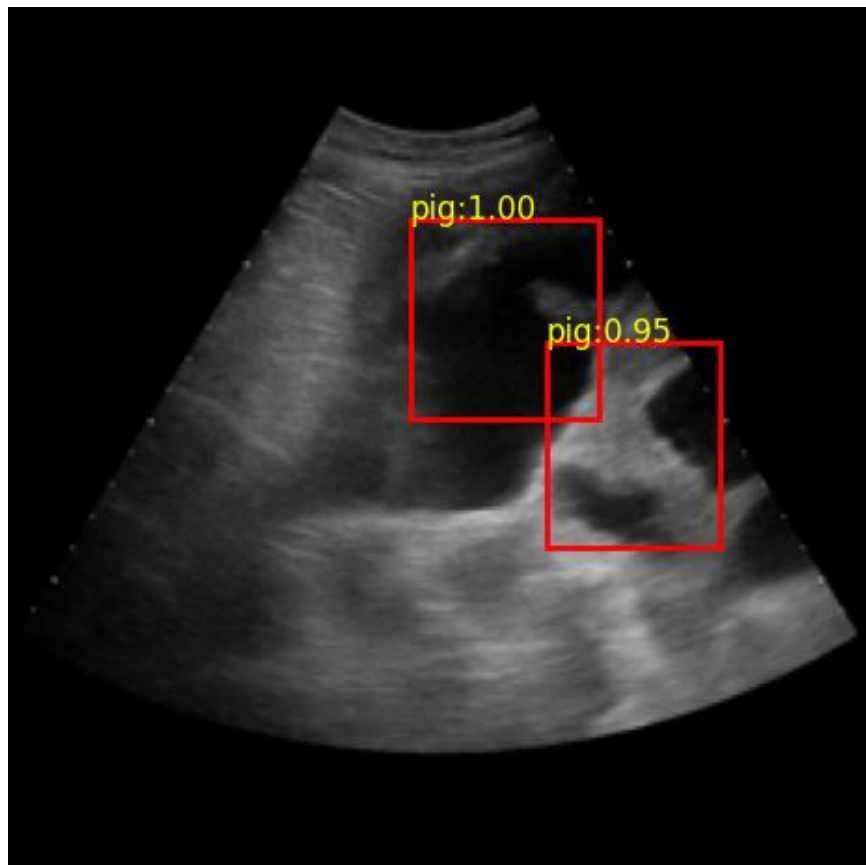
Check item	What to verify
Images and labels	Matching filenames · overlay box coordinates on images
Coordinates and preprocessing	Normalized vs. pixel coordinates · transform boxes with augmentation
Training settings and model	Check learning rate, trainable weights, input resolution

Check the training path on a small dataset

Use ~10 well-labeled images; check loss decrease and predictions

Do not assume a cause; check data, code and settings step by step

Compare ground truth with detections to identify error types



| Red box

Where the model says it found the target

| Number

Closer to 1 means more confident. Usually drop below 0.5

| Only visible by eye

Wrong location? One object split into two?
Check for missed overlapping objects

Good metrics alone cannot show what went wrong without viewing result images

Precision and recall break down detection performance

Compare predicted and ground-truth boxes by position and class to separate TP, FP, FN

Recall

Share of real targets that were found

Lower value, more missed detections

Precision

Share of detections that are real targets

Lower value, more false detections

When adjusting the detection threshold

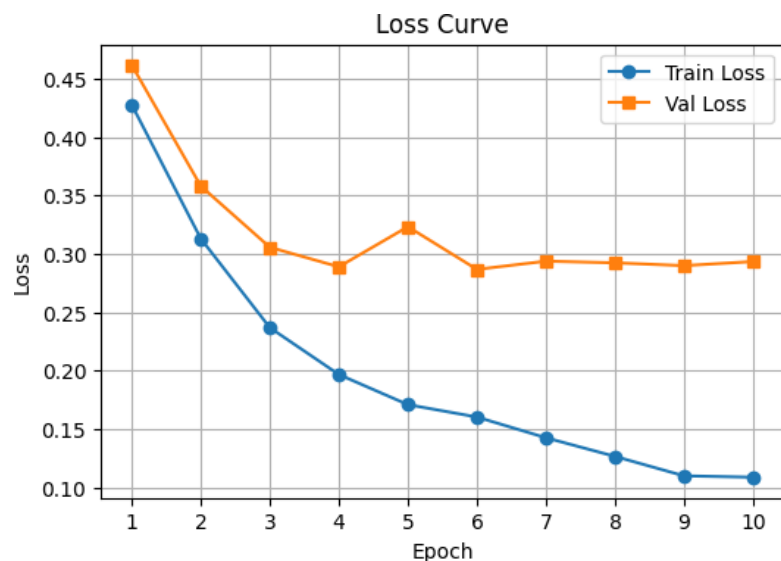
Lowering the score threshold keeps or raises detections and recall
If extra detections include false positives, precision may drop

Pregnancy-status metrics and detection precision/recall are evaluated separately

Practice log comparing different settings

Saved curves after baseline training, augmentation, and setting changes

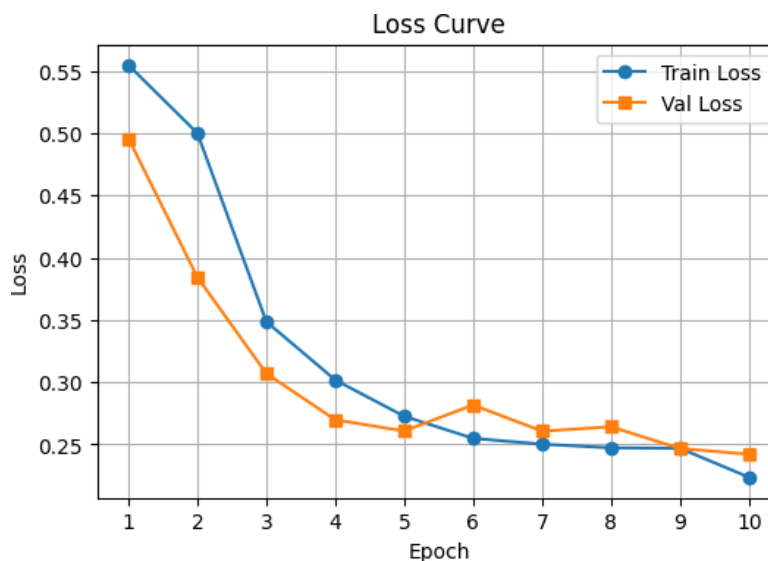
Baseline training



Validation loss plateaus

Output P / R 0.815 / 0.857

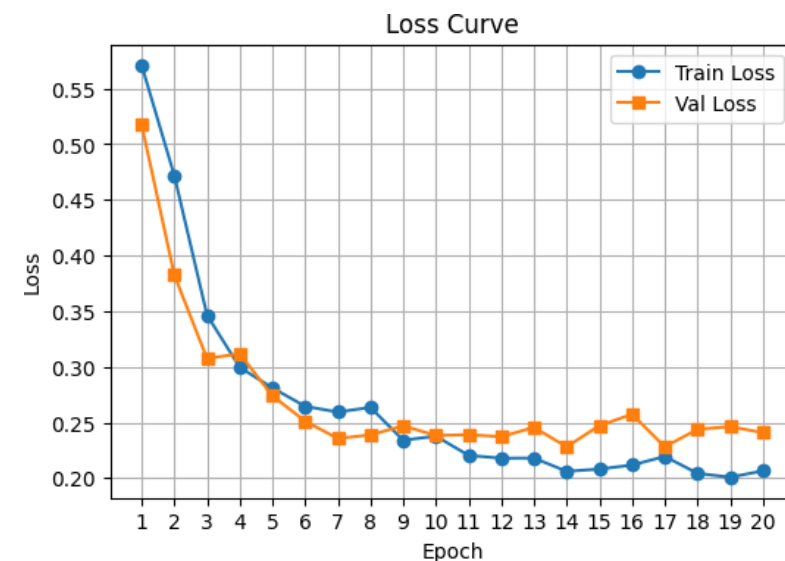
With data augmentation



Train-validation gap narrows

Output P / R 0.640 / 0.905

Further setting changes



Log of 20 epochs

Output P / R 0.678 / 0.922

Before re-validating, check run logs, evaluation code and conditions, then judge improvement

Apply the saved model to new images and keep the results

After training, save weights (.pt) and apply the same model to new inputs

```
from ultralytics import YOLO

# path of the uploaded best weights
best = YOLO("/content/best.pt")
best.predict(
    source="/content/new_image.bmp",
    conf=0.25, save=True
)
```

What to keep

best.pt: selected model
Run settings and data list
Detection result images

conf sets the display threshold
Tune it on validation data

Final evaluation: apply the model to held-out data and compare with ground truth

Judge applicability to your own research with independent data and real error cases

Criteria the researcher must decide when using AI

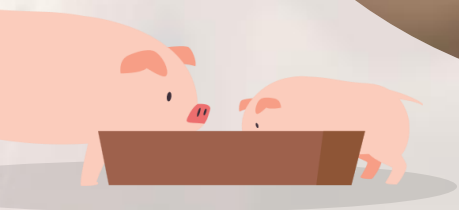
Beyond generating and running code, validate the data and the evaluation

Researcher's decision	Question in this practice
Labeling criteria	• What to find, and how far to label as ground truth
Data split	• Which animals or recordings to hold out for evaluation
Result evaluation	• Which metric shows whether validation improved • On what basis to compare misses and false detections



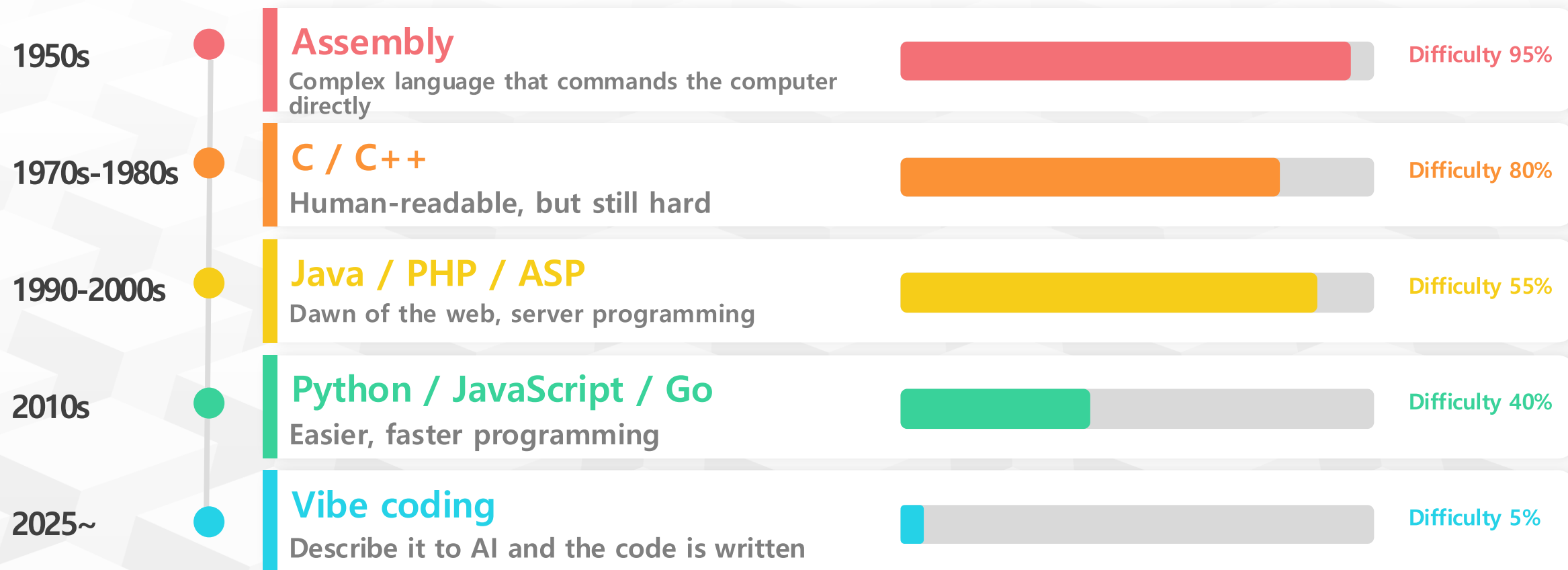
Step 3

Understanding vibe coding



Programming languages

Programming languages evolved
around humans are shifting to vibe
coding with AI



What is vibe coding?

The user describes the requirements in natural language,
and AI generates working code

How coding is changing

Traditional coding



- 1 Learn a language → Python, JavaScript, ...
- 2 Learn concepts → variables, functions, conditionals, OOP, ...
- 3 Try a project → endless errors, googling

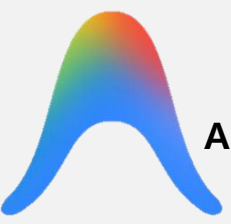
Vibe coding



- 1 Describe what you want
- 2 AI generates code
- 3 Run and check
- 4 Request changes

Coding tools

AI coding editors



Google Antigravity



Claude Code



Cursor

Prototyping tools



Google AI Studio



Lovable



Replit

What is an LLM?

✓ Large Language Model

LLM = AI trained on huge amounts of text

Trained on vast internet text (books, wikis, code) to predict the next word

Vibe coding

You describe it in words, the LLM writes the code

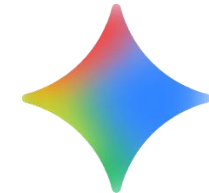
Major LLMs



Claude(Anthropic)



GPT(OpenAI)



Gemini(Google)



Groq(xAI)

How an LLM

works

My question
(prompt)



LLM
parses/understands



Finds answer in
learned knowledge



Generates code/answer



Outputs result

Vibe coding

checkpoints

- Same question, different phrasing, different result → prompt engineering
- LLMs are not 100% accurate → hallucinations are possible
- Each LLM has its own strengths → Claude is strong at coding

What is a prompt?

✓ A question/command sent to AI = the language of vibe coding

Bad prompt

"Write statistical analysis code"

"Fix the error"

"That's not it, try again"

→ AI can't tell what you want; results all over the place

Good

"Write Python code for multiple linear regression. Data is a pandas DataFrame; y: dependent variable, x1: independent variable, x2: independent variable ~ "

"TypeError: Cannot read Property" — read this error message and the code, then fix it

"Normalize variables, run residual analysis, then let me save regression diagnostic plots"

→ Specific wording, AI understands

Prompt checkpoints

1. Be specific.

2. Give context.

3. Give examples.

4. Ask step by step

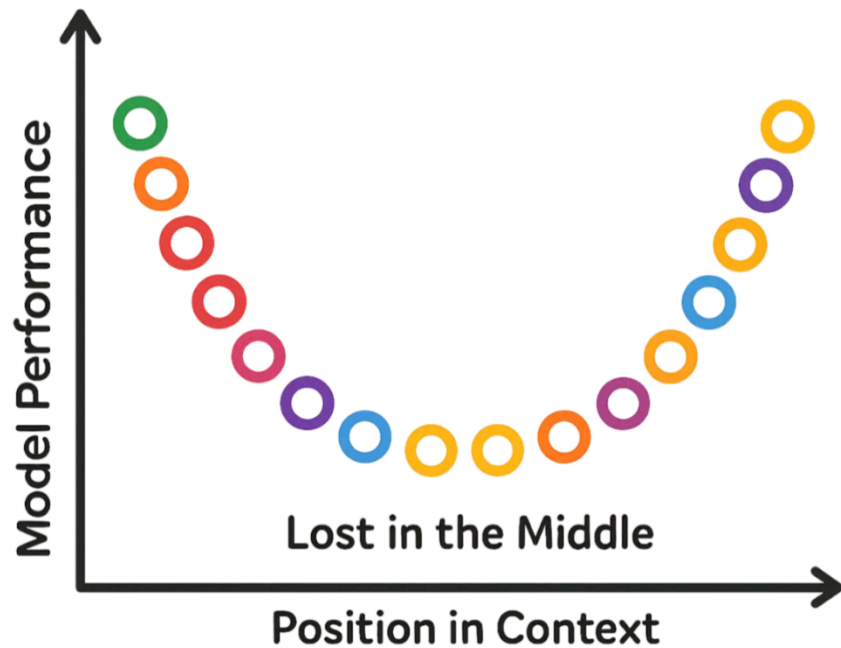
5. Assign a role.

Context engineering

✓ Designing the right context for AI

Prompt engineering

Single-interaction skill focused on writing good commands for AI



Context engineering

Designing the whole information environment a model needs to answer

→ Covers not just prompts but RAG, conversation memory, APIs

- ✓ With long context, model performance drops for material in the middle
 - The longer the conversation, the longer the context
- ✓ Plain prompting cannot manage dynamic, changing task state
 - Hard to access changing data or update from tool (API) call results

A new approach to improving LLM performance

Which information, in which order, makes a large difference in performance. Not just asking well, but designing context: relevant data, search results, and more



Step 4 Research applications

Problem · Field challenge

Training one model means a person clicking tens of thousands of points

Data – volume needed for training

1,000+ annotated images per class

'Object' = what the worker wants to detect (in pig photos, each individual pig)

Source: Google Cloud Vertex AI



Manual labeling

20 min

Average per image · polygon basis

Must be repeated for hundreds to thousands of images

| Labeling burden

Boxes and polygons drawn by hand → enormous time cost

| Uneven quality

Label variance by worker skill → affects model performance

| Task difficulty

Overlap, occlusion, lighting, complex backgrounds make object borders hard to judge

| Lower research efficiency

Repetitive work eats research time · repeated for every new dataset

Insight

Turn 'marking objects' into 'reviewing objects'

BEFORE

Mark everything by hand

Worker marks object regions point by point on a blank screen



AFTER

AI drafts, human reviews

AI first-pass
detection



Worker reviews and
corrects

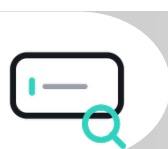


Final label saved

Why switch to 'review'

Drawing time becomes review time

Tens of thousands of objects per model — review AI drafts instead of drawing



AI labeling assistant expected to cut time, improve dataset efficiency and label consistency

How it works · Pipeline

From one line of text to a JSON file

Step 1

Load images

Single / folder
mode

Step 2

Enter class text

e.g. "pig", "apple"

Step 3

SAM3 auto- detection

Auto-detects position and
region

Step 4

Manual review and edit

Polygon correction

Step 5

Save JSON

Saves coordinates, classes,
etc.

Mode

Single image · folder batch

- ✓ Detect many images in a folder at once
- ✓ Choose skip or overwrite
- ✓ Live progress display and mid-run stop

Stable

Operational stability

- ✓ Model loading and detection on separate threads prevent UI freezing
- ✓ Confirmation dialog before saving prevents data overwrite

Tools used

Ultralytics SAM3

Python · tkinter

OpenCV · PIL

Claude

Live Demo

Technical points

- ✓ Conf · IoU · Epsilon retuned instantly
- ✓ JSON independent of ratio, coordinates, resolution
- ✓ Multithreading for uninterrupted detection

1 Set file paths

Model file, working file

2 Enter class name

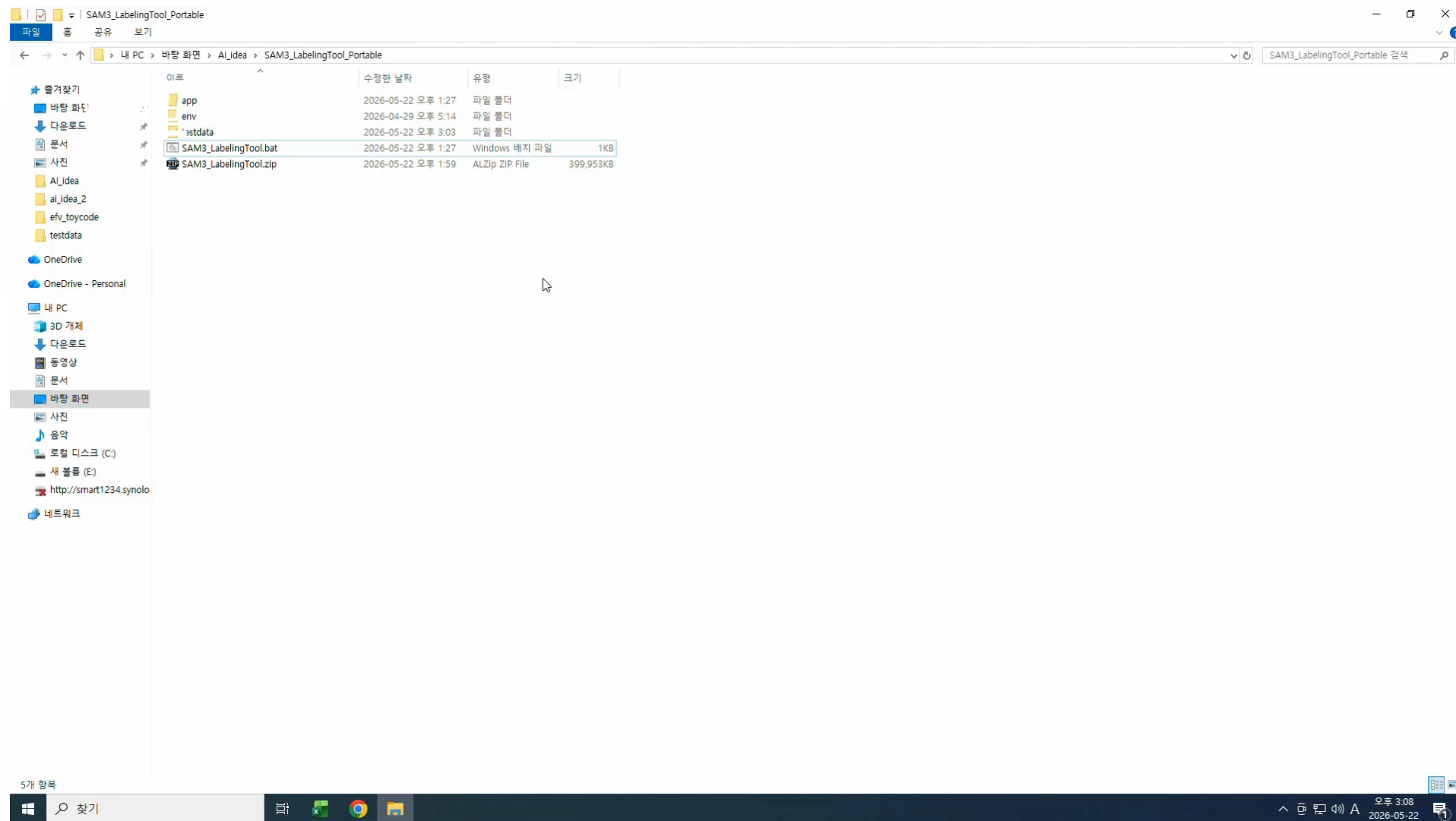
e.g. "pig", "apple"

3 Run detection (SAM)

Polygons generated automatically

4 Correct, then save

Edit tools → Ctrl+S



Not an estimate — a measured result

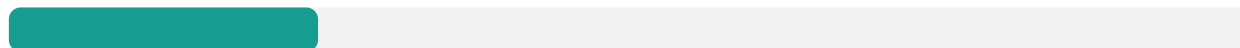
80%

Before AI tool



20 min / image

After AI tool



2~5 min / image

Less working time

Instance segmentation · based on 12 pigs

Less fatigue

Eases repetitive manual work

Consistent quality

AI provides the frame

Lower barrier ↓

Usable at once, even by non-experts

Scale · Spread within the organization

Change the class name, use it anywhere

"Pig"

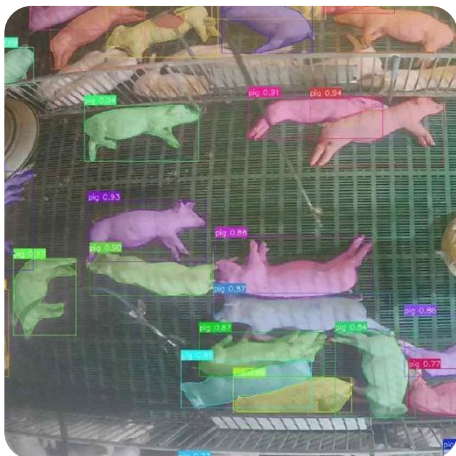


"Cow"

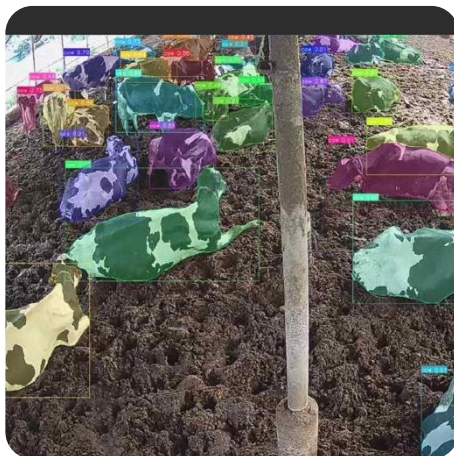
"Apple"

"Eggplant"

Validation results



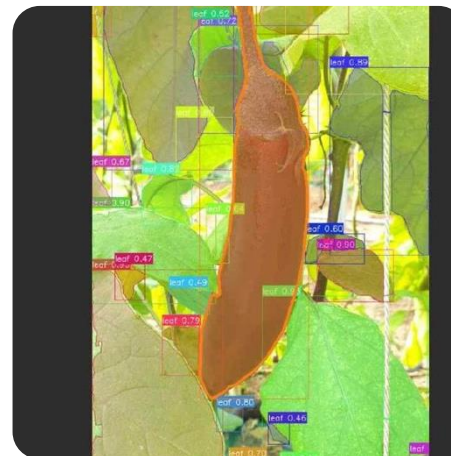
<Pig entered>



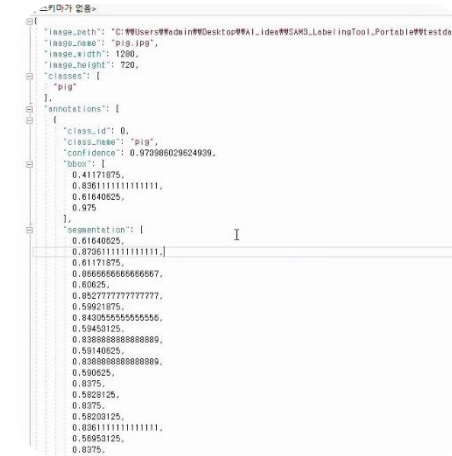
<Cow entered>



<Apple entered>



<Eggplant, Leaf entered>



<JSON saved>

Applicable to detection research at once

In object detection model research, just change the class name for general use

Offline deployment

Virtual environment and libraries packaged together → easy to deploy, no setup

Thank you